

CMPUT 328 — Assignment 2

Convolutional neural networks and vision transformers

Total weight	9% of the course grade — Part 1 (CNNs) 4% + Part 2 (ViT) 5%
Deadline	8 October 2026, 11:59 PM (MST)
Submission	1. Completed Colab notebook with all required code, outputs, and plots visible. 2. A 1–2 page PDF report. Include your name and student ID on both files. If AI assistance is used, document it in the report.

Goal

Build and train image classification models on CIFAR-10, and analyze their training dynamics, evaluation results, and errors. You will work with convolutional neural networks (CNNs) and Vision Transformers (ViTs), comparing a ViT trained from scratch with an ImageNet-pretrained ViT fine-tuned using LoRA (Low-Rank Adaptation). You will also investigate the effects of architecture, pretraining, amount of training data, and fine-tuning strategy on accuracy and training cost.

Deliverables

- Completed Colab notebook with code, outputs, plots, and short comments.
- A 1–2 page PDF report summarizing your findings.
- Include your name and student ID on both files.
- If you use Gemini or another AI assistant, document your usage in the report.

Part 1 (4%) — CIFAR-10 Classification with CNNs

A **Convolutional Neural Network (CNN)** is a neural network designed to work with data that has a spatial structure, such as images. Unlike a fully connected neural network, a CNN does not immediately flatten an image into a one-dimensional vector. Instead, it processes the image while preserving its spatial structure.

For example, a CIFAR-10 image has dimensions: $32 \times 32 \times 3$, where 32×32 represents the image height and width, and 3 represents the RGB colour channels. CNNs keeps the input image in its spatial form while applying convolutional filters. This allows the network to learn from local regions and preserve spatial relationships between nearby pixels. For this assignment:

1. Implement a small convolutional neural network (CNN) in PyTorch for image classification on CIFAR-10. Your CNN should use convolutional layers, ReLU activations, and pooling layers. Do not flatten the image at the input.
2. Prepare the CIFAR-10 dataset using `torchvision.datasets.CIFAR10`. Split the official 50,000 training images into a **45,000-image training** set and a **5,000-image validation** set, and keep the 10,000-image test set for final evaluation. Normalize the images, and set all random seeds (`torch.manual_seed(328)`, `random_seed(328)`, etc.).
3. Train the CNN for 30 epochs using either the Adam or SGD optimizer. Track and report the training and validation loss and accuracy for each epoch. Do **not** evaluate on the test set during training. Perform a training sanity check by overfitting a small batch of approximately 128 images. Use this to verify that your model and training loop are working correctly.
4. Verify the dimensions of the tensors after each CNN layer. You can pass a sample batch through the network and print the tensor shape after each layer (conv, activation, pooling etc.). Explain how the

spatial dimensions change as the image passes through the convolution and pooling layers.

5. Plot the training curves, including loss and accuracy across epochs. Briefly discuss the model's training behaviour and whether the model appears to be learning effectively.
6. Evaluate the trained CNN on the test/validation set and report its classification accuracy. As a guideline, approximately 55%-65% accuracy after about 10 epochs on the subset is considered a reasonable baseline.
7. Visualize example predictions by displaying several CIFAR-10 images along with their true and predicted classes. Include examples of both correct and incorrect predictions.
8. Briefly interpret your results. Discuss what you observe from the loss/accuracy curves, example predictions, and the model's final accuracy.

Part 2 (5%) — CIFAR-10 Classification with a ViT and LoRA Fine-Tuning

Implement and train a compact Vision Transformer (ViT) from scratch on CIFAR-10. Then fine-tune an ImageNet-pretrained ViT on CIFAR-10 using LoRA (Low-Rank Adaptation), and compare the two approaches in terms of accuracy and training cost.

1. Data Pipeline

Load CIFAR-10 using `torchvision.datasets.CIFAR10`, normalize the data, and create a training set (45,000 images), a validation set (5,000 images), and a test set (the official 10,000 test images). Document your split and set all random seeds (e.g., `torch.manual_seed(328)`, `random.seed(328)`, `np.random.seed(328)`).

2. Vision Transformer from Scratch

Implement a compact ViT for CIFAR-10. Your model must split each image into patches and tokenize them; do not flatten the entire image at the input. Your implementation should include a patch embedding, a learnable [CLS] token, positional embeddings, a stack of Transformer encoder blocks, and a classification head. Keep the model small enough to train quickly (e.g., patch size 4 on the native 32×32 images).

Before full training, perform two sanity checks: (a) overfit a tiny batch (e.g., 128 images) to confirm the training loop works, and (b) print tensor shapes after each stage of the model.

Train for 20–30 epochs with Adam or AdamW and a reasonable learning rate (e.g., 1e-4 to 3e-4).

3. Training Dynamics and CNN Comparison

Plot training loss vs. validation loss, and training accuracy vs. validation accuracy. Interpret the resulting curves. Re-run (or reuse) your CNN with the same data split and a similar epoch budget, and report its test accuracy. Briefly discuss why the ViT trained from scratch does or does not outperform the CNN on this dataset.

4. Loading an ImageNet-Pretrained ViT

Load a ViT pretrained on ImageNet ([e.g., google/vit-base-patch16-224-in21k](https://huggingface.co/google/vit-base-patch16-224-in21k) from Hugging Face). Replace its classification head with a new 10-class head. Resize CIFAR-10 images to 224×224 and apply the normalization expected by the pretrained model.

As a baseline, freeze the entire backbone and train only the new head (linear probing) for 1–2 epochs, then report the test accuracy.

5. LoRA Fine-Tuning

Apply LoRA to the pretrained ViT using the `peft` library. Inject LoRA adapters into the attention projections (e.g., query and value for the Hugging Face ViT, or qkv for timm), with a rank such as $r = 8$ or 16. Keep the new classification head fully trainable. Fine-tune on CIFAR-10 for 2–3 epochs.

Report the number of trainable parameters (and their percentage of the total), time per epoch, peak GPU memory (`torch.cuda.max_memory_allocated()`), and test accuracy. For comparison, also run full fine-tuning of the same pretrained

model with the same epoch budget and report the same four quantities. If full fine-tuning does not fit in memory, reduce the batch size or train on a subset, and state what you changed.

6. Results Comparison

Summarize all models in a single table: ViT from scratch, CNN, linear probe, LoRA fine-tuning, and full fine-tuning. Include test accuracy, trainable parameters, time per epoch, and peak GPU memory for each. Compute and plot a confusion matrix on the validation set for both the ViT from scratch and the LoRA-fine-tuned ViT, and discuss how the confusion patterns differ. Display a grid of at least 16 misclassified validation images from the LoRA model, showing the predicted and true labels for each image.

7. Controlled Experiment

Make one deliberate change to the fine-tuning setup and re-run it. Examples include the LoRA rank (e.g., $r = 4$ vs. $r = 32$), the target modules (attention only vs. attention + MLP layers), the learning rate, or the amount of training data (e.g., 10% of the training set for both the from-scratch ViT and the LoRA model). State what you expect to happen before making the change, and report what actually happened afterward.

Report

Write a 1–2 page PDF report summarizing your findings from both parts of the assignment:

PART 1

- Clarify the transformations in spatial dimensions (height and width) alongside channel depth across all convolutional and pooling layers, detailing the impact of filter size, stride, and padding configurations.
- Discuss the training behaviour. Is the model learning? Is there evidence of underfitting or overfitting?
- Using the example predictions, discuss which classes the model confuses and suggest reasons, such as visual similarity or low resolution. Relate this to the curves and the final test accuracy.

PART 2

- Where does the train/val gap open up? Does validation loss plateau or overfit?
- Discuss why the scratch ViT does or doesn't outperform the CNN.
- Discuss how the error patterns differ between scratch ViT and LoRA using the confusion matrix.
- Comment on what the LoRA model's errors have in common using misclassified images.
- State the hypothesis before applying the controlled experiment, then explain what actually happened (especially if it didn't match).

Performance Expectations

The following performance targets are used for full marks on the training components. These targets are reachable with the notebook's default FAST_MODE settings.

Model	Ideal validation accuracy
Convolutional neural networks (CNNs)	$\geq 80\%$
Vision Transformers((from scratch)	$\geq 75\%$

Runtime (FAST_MODE)

The notebook defines a FAST_MODE flag that trains on a random subset of each training set so the assignment runs quickly on Colab. It is **enabled by default**. Keep the same setting for every experiment you compare, and state in your report which setting you used. **We would recommend initially running with fast mode on and then running the notebook with fast mode off and report those numbers**

Rubric

Each rubric criterion is graded independently. Full marks are awarded when **all** requirements within a criterion are satisfied; otherwise, the criterion receives zero. There are no partial marks within an individual criterion.

Part 1 — CNN (4%)

Component	Weight	Criteria
1. CNN Architecture	0.5	Uses conv, ReLU and pooling layers; input stays $N \times 3 \times 32 \times 32$; flattening only before the classifier
2. Data Pipeline	0.5	Provided 45k / 5k / 10k split used unchanged; normalization applied; seeds set
3. Sanity check and tensor shapes	1.0	Overfits one 128-image batch to $\geq 95\%$ accuracy (or loss < 0.1) within 200 iterations; result reported. Shape printed after every layer; spatial changes explained using kernel size, stride and padding.
4. Training and curves	0.5	5–10 epochs with Adam or SGD; per-epoch train/val loss and accuracy plotted; behaviour discussed
5. Performance	0.5	$> 65\%$ validation accuracy with FAST_MODE = False
6. Example Predictions	0.5	Grid shows true and predicted labels, with both correct and incorrect examples
7. Interpretation	0.5	Discusses error patterns and relates them to the curves and final accuracy

Part 2 — ViT (5%)

Component	Weight	Criteria
1. Data Pipeline	0.4%	45,000/5,000/10,000 split correctly drawn from the official CIFAR-10 train/test sets and documented; all random seeds set (torch.manual_seed, random.seed, np.random.seed, e.g. 328).
2. Vision Transformer from Scratch	0.8%	Image is split into patches and tokenized (not flattened whole); includes patch embedding, learnable [CLS] token, positional embeddings, a stack of Transformer encoder blocks, and a classification head. Both sanity checks present: tiny-batch overfit to near-100% accuracy, and tensor shapes printed after each stage. Trained 20–30 epochs with Adam/AdamW at a reasonable LR.
3. Training Dynamics and CNN Comparison	0.6%	Loss/accuracy plots included. CNN re-run on the identical split and a comparable epoch budget; test accuracy reported.
4. Loading an ImageNet-Pretrained ViT	0.4%	ImageNet-pretrained ViT loaded correctly with a new 10-class head; images resized to 224×224 and normalized per the pretrained model's stats. Backbone fully frozen for the linear probe (1–2 epochs); test accuracy reported.
5. LoRA Fine-Tuning	0.8%	LoRA adapters correctly injected into attention projections via peft, with a stated rank; classification head kept trainable. Full fine-tuning run with the same epoch budget. All four quantities reported for both: trainable params (and %), time/epoch, peak GPU memory, test accuracy.
6. Results Comparison	0.6%	Summary table across all models. Confusion matrices for both scratch ViT and LoRA. Grid of ≥ 16 misclassified LoRA images with predicted/true labels.
7. Controlled Experiment	0.4%	One deliberate, clearly-stated change, run and reported.
Report	1.0%	Analysis is present and correct for: training-curve interpretation (train/val gap, over/underfitting), why the ViT does/n't outperform the CNN, how the confusion-matrix patterns differ between scratch ViT and LoRA, what the misclassified LoRA images have in common, and the controlled experiment's hypothesis vs. actual outcome.

